URLSCAN.IO

Bug Bounty Hunting Guide

Target-Based Recon | Historical Scans | API Automation

Endpoint Extraction | Tech Fingerprinting | Workflow Integration

2025 Edition

Table of Contents

1. Introduction to URLScan.io

URLScan.io is a free web-based sandbox service that scans URLs and provides detailed information about the page content, technologies, network requests, and behavior. For bug bounty hunters, it serves as both a passive reconnaissance tool (searching historical scans) and an active analysis tool (submitting URLs for live scanning).

> **What URLScan.io Provides**
>
> Page screenshots, DOM content, network requests (HAR), JavaScript file links, technology detection (Wappalyzer), certificate information, IP geolocation, ASN data, server headers, cookie analysis, and chain of redirects. All scans are public by default and searchable by anyone.

Feature	Bug Bounty Value
Historical Scans	Discover past URLs that may no longer be publicly linked
Network Requests	Extract API endpoints, tracking pixels, third-party calls
Technology Detection	Identify framework versions for CVE targeting
Certificate Info	Find alternative domain names (SANs) in certificates
IP & ASN	Map infrastructure and discover co-located services
DOM Snapshot	Analyze client-side rendered content and hidden elements
Redirects	Map redirect chains and identify open redirects

> **Public Scan Warning**
>
> Scans submitted to URLScan.io are **publicly visible** by default. Anyone can search and view your scan results, including the target URL, page content, and cookies. For sensitive reconnaissance, consider using the private tag option or avoid scanning authenticated/internal endpoints.

2. Search Operators & Query Syntax

2.1 Domain & Page Filters

Operator	Description	Example
`domain:`	Exact domain match	`domain:target.com`
`page.domain:`	Page domain (after redirects)	`page.domain:target.com`
`task.url:`	Submitted URL pattern	`task.url:*.target.com/*`
`page.url:`	Final page URL	`page.url:*api*`
`page.title:`	Page title contains	`page.title:"admin"`

```
# Search all scans for a specific domain
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com" | jq '.results[]'

# Search for a specific subdomain
curl -s "https://urlscan.io/api/v1/search/?q=domain:api.target.com" | jq '.results
[].page.url'

# Search by page title (find admin panels)
curl -s "https://urlscan.io/api/v1/search/?q=page.title:admin" | jq '.results[]'

# Search for specific URL patterns
curl -s "https://urlscan.io/api/v1/search/?q=page.url:*admin*" | jq '.results[].pag
e.url'

# Combine operators
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com AND page.title:login"
| jq '.results[]'
```

2.2 Content & Technology Filters

Operator	Description	Example
`page.server:`	Server header value	`page.server:nginx`
`page.status:`	HTTP status code	`page.status:200`
`page.mime:`	Content type	`page.mime:text/html`
`page.ip:`	IP address	`page.ip:1.2.3.4`
`page.asn:`	ASN number	`page.asn:AS12345`
`page.asnname:`	ASN organization name	`page.asnname:Amazon`
`page.country:`	Country code	`page.country:US`
`page.city:`	City name	`page.city:London`
`hash:`	SHA256 of page content	`hash:abc123...`

```
# Find all scans for a specific IP
curl -s "https://urlscan.io/api/v1/search/?q=page.ip:52.84.123.45" | jq '.results[]'

# Find all domains on same ASN (infrastructure mapping)
curl -s "https://urlscan.io/api/v1/search/?q=page.asn:AS16509" | jq '.results[].pag
e.domain' | sort -u

# Search by server type
curl -s "https://urlscan.io/api/v1/search/?q=page.server:Apache AND domain:target.co
m" | jq '.results[]'

# Find redirect chains to a specific domain
curl -s "https://urlscan.io/api/v1/search/?q=page.domain:evil.com" | jq '.results[].
task.url'

# Find all login pages on a target
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com AND page.title:login"
| jq '.results[].page.url'
```

2.3 ASN, IP & Network Filters

```
# Find all domains on same IP (virtual host discovery)
curl -s "https://urlscan.io/api/v1/search/?q=page.ip:1.2.3.4" | jq '.results[].page.
domain' | sort -u

# Count unique domains on an IP
curl -s "https://urlscan.io/api/v1/search/?q=page.ip:1.2.3.4&size=10000" | \
  jq -r '.results[].page.domain' | sort -u | wc -l

# Find all targets on AWS us-east-1
curl -s "https://urlscan.io/api/v1/search/?q=page.asn:AS14618 AND page.city:Ashburn"
| \
  jq -r '.results[].page.domain' | sort -u

# Find targets by country
curl -s "https://urlscan.io/api/v1/search/?q=page.country:DE AND page.title:admin" |
\
  jq -r '.results[].page.url'
```

> **Pagination Tip**
>
> URLScan.io search API returns 100 results per page by default. Use `&size=10000` for maximum results in a single query. For larger datasets, paginate with `search_after` parameter from the last result sort key.

3. Reconnaissance Techniques

3.1 Subdomain Discovery

URLScan.io maintains a history of scanned URLs. By searching for all pages where the submitted URL or final page domain matches your target, you can discover subdomains that may not appear in certificate transparency logs or DNS records.

```
# Extract all unique subdomains from URLScan
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com&size=10000" | \
  jq -r '.results[].page.domain' | sort -u

# Extract all task URLs (submitted URLs, includes redirects)
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com&size=10000" | \
  jq -r '.results[].task.url' | grep -oE '[a-zA-Z0-9.-]+\.target\.com' | sort -u

# Extract all domains from certificate SANs in scans
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com&size=10000" | \
  jq -r '.results[].page.certinfo?.subjectAltName[]?' 2>/dev/null | sort -u

# Full subdomain extraction pipeline
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com&size=10000" | \
  jq -r '.results[] | [.task.url, .page.url, .page.domain] | .[]' | \
  grep -oE '[a-zA-Z0-9][a-zA-Z0-9.-]*\.target\.com' | sort -u > urlscan_subdomains.t
xt

# Combine with other subdomain sources
cat urlscan_subdomains.txt amass_subdomains.txt subfinder_subdomains.txt | \
  sort -u > all_subdomains.txt
```

URLScan + httpx Live Check Pipeline

```
# Extract from URLScan and verify live hosts
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com&size=10000" | \
  jq -r '.results[].page.domain' | sort -u | \
  httpx -silent -mc 200,301,302 -o live_hosts.txt
```

Passive urlscan httpx

3.2 URL & Endpoint Extraction

Every URLScan result contains a list of all network requests made during page loading. This includes API calls, third-party requests, CDN resources, tracking pixels, and WebSocket connections. Extracting these reveals hidden endpoints and dependencies.

```
# Get a specific scan result and extract all URLs
curl -s "https://urlscan.io/api/v1/result/scan-uuid-here/" | \
  jq -r '.data.requests[]?.request?.url' | sort -u

# Extract API endpoints from a scan
curl -s "https://urlscan.io/api/v1/result/scan-uuid-here/" | \
  jq -r '.data.requests[]?.request?.url' | grep -i "api" | sort -u

# Extract all JavaScript files from a scan
curl -s "https://urlscan.io/api/v1/result/scan-uuid-here/" | \
  jq -r '.data.requests[]?.request?.url' | grep '\.js$' | sort -u

# Extract all GraphQL or API calls
curl -s "https://urlscan.io/api/v1/result/scan-uuid-here/" | \
  jq -r '.data.requests[]?.request?.url' | grep -E 'graphql|api|rest|v[0-9]' | sort
-u

# Automate for multiple scans - extract URLs from all recent target scans
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com&size=100" | \
  jq -r '.results[]._id' | while read uuid; do
    echo "=== Scan: $uuid ==="
    curl -s "https://urlscan.io/api/v1/result/$uuid/" | \
      jq -r '.data.requests[]?.request?.url'
  done | sort -u > all_urls.txt
```

Extraction Target	JQ Filter
All request URLs	`.data.requests[]?.request?.url`
Response status codes	`.data.requests[]?.response?.status`
Response headers	`.data.requests[]?.response?.headers`
Cookies	`.data.cookies[]?.name, .data.cookies[]?.domain`
Console messages	`.data.console[]?.message`
Links/domains	`.lists.linkDomains[]?`
Certificates	`.page.certinfo`
Technologies	`.meta.processors?.wappalyzer?.data?.applications[]?.name`

3.3 Technology Stack Identification

```
# Extract detected technologies for a scan
curl -s "https://urlscan.io/api/v1/result/scan-uuid/" | \
  jq -r '.meta.processors?.wappalyzer?.data?.applications[]? | "\(.name) - \(.versio
n // "unknown")"'

# Find all sites using a specific technology (e.g., WordPress)
curl -s "https://urlscan.io/api/v1/search/?q=page.title:WordPress&size=100" | \
  jq -r '.results[].page.domain' | sort -u

# Find all React apps on a target
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com" | \
  jq -r '.results[] | select(.meta.processors?.wappalyzer?.data?.applications[]?.nam
e == "React") | .page.url'

# Find all Laravel sites (check server header + technology)
curl -s "https://urlscan.io/api/v1/search/?q=page.server:nginx AND domain:target.co
m" | \
  jq -r '.results[].page.url'
```

> **Technology Hunting Tip**
>
> Search URLScan for known-vulnerable framework versions. For example, find all sites running Apache Struts 2 or old WordPress versions by combining technology detection with public CVE disclosures.

4. API Automation

4.1 Search API

```
# Basic search API structure
GET https://urlscan.io/api/v1/search/?q={query}&size={limit}&offset={offset}

# Search with all parameters
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com&size=100&offset=0" |
jq

# Extract just the result UUIDs for processing
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com&size=100" | \
  jq -r '.results[]._id'

# Search for specific time range
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com&search_after=16725312
00" | jq

# Search for scans from the last 24 hours
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com AND date:>now-24h" |
jq

# Paginate through large result sets
offset=0
while true; do
  results=$(curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com&size=100&
offset=$offset" | jq '.results')
  count=$(echo "$results" | jq 'length')
  if [ "$count" -eq 0 ]; then break; fi
  echo "$results" | jq -r '.[].page.url'
  offset=$((offset + 100))
done
```

4.2 Live Scanning API

```
# Submit a URL for scanning (public)
curl -X POST "https://urlscan.io/api/v1/scan/" \
  -H "Content-Type: application/json" \
  -d '{"url": "https://target.com", "public": "on"}'

# Submit with custom tags
curl -X POST "https://urlscan.io/api/v1/scan/" \
  -H "Content-Type: application/json" \
  -d '{"url": "https://target.com", "customagent": "Mozilla/5.0", "referer": "http
s://google.com", "tags": ["bugbounty","target"]}'

# Submit with API key (higher rate limits)
curl -X POST "https://urlscan.io/api/v1/scan/" \
  -H "Content-Type: application/json" \
  -H "API-Key: YOUR_API_KEY" \
  -d '{"url": "https://target.com"}'

# Submit and poll for result
scan_result=$(curl -s -X POST "https://urlscan.io/api/v1/scan/" \
  -H "Content-Type: application/json" \
  -d '{"url": "https://target.com"}')
uuid=$(echo "$scan_result" | jq -r '.uuid')
echo "Scan UUID: $uuid"

# Poll every 10 seconds until complete
while true; do
  status=$(curl -s "https://urlscan.io/api/v1/result/$uuid/" | jq -r '.task.status')
  echo "Status: $status"
  if [ "$status" = "done" ] || [ "$status" = "null" ]; then break; fi
  sleep 10
done

# Get the full result
curl -s "https://urlscan.io/api/v1/result/$uuid/" | jq
```

4.3 Result Parsing

```
# Extract all network requests with status and type
curl -s "https://urlscan.io/api/v1/result/$uuid/" | \
  jq -r '.data.requests[]? | "\(.response?.status // "??") \(.request?.method // "GE
T") \(.request?.url)"'

# Extract cookies set during the scan
curl -s "https://urlscan.io/api/v1/result/$uuid/" | \
  jq -r '.data.cookies[]? | "\(.name)=\(.value); Domain=\(.domain)"'

# Extract console errors (often reveal internal paths)
curl -s "https://urlscan.io/api/v1/result/$uuid/" | \
  jq -r '.data.console[]?.message' | grep -i "error\|fail\|404\|500"

# Extract certificate information
curl -s "https://urlscan.io/api/v1/result/$uuid/" | \
  jq '.page.certinfo | {subject,issuer,subjectAltName,validFrom,validTo}'

# Extract all link domains (third-party resources)
curl -s "https://urlscan.io/api/v1/result/$uuid/" | \
  jq -r '.lists.linkDomains[]?.domain' | sort -u

# Extract detected technologies
curl -s "https://urlscan.io/api/v1/result/$uuid/" | \
  jq -r '.meta.processors?.wappalyzer?.data?.applications[]? | "\(.name): \(.version
// "N/A")"'

# Extract response headers from main page
curl -s "https://urlscan.io/api/v1/result/$uuid/" | \
  jq '.data.requests[]? | select(.request?.url | contains("target.com")) | .respons
e?.headers'
```

5. Target-Based Hunting

5.1 Finding Exposed Admin Panels

```
# Search for admin-related page titles on target
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com AND page.title:admin"
| \
  jq -r '.results[].page.url'

# Search for login pages
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com AND page.title:login"
| \
  jq -r '.results[].page.url'

# Search for specific admin paths
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com AND page.url:*admin*"
| \
  jq -r '.results[].page.url'

# Search for WordPress admin
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com AND page.url:*wp-admi
n*" | \
  jq -r '.results[].page.url'

# Search for phpMyAdmin
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com AND page.url:*phpmyad
min*" | \
  jq -r '.results[].page.url'

# Search for Grafana, Kibana, Jenkins
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com AND page.title:grafan
a" | \
  jq -r '.results[].page.url'

curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com AND page.title:jenkin
s" | \
  jq -r '.results[].page.url'

curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com AND page.url:*kibana
*" | \
  jq -r '.results[].page.url'
```

> **Responsible Disclosure**
>
> If you discover exposed admin panels, internal tools, or unauthenticated dashboards via URLScan.io, **do not attempt to log in or access them**. Report them immediately as information disclosure or misconfiguration findings.

5.2 Discovering API Endpoints

```
# Search for API-related paths
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com AND page.url:*api*" |
\
  jq -r '.results[].page.url'

# Search for GraphQL endpoints
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com AND page.url:*graphql
*" | \
  jq -r '.results[].page.url'

# Search for swagger docs
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com AND page.url:*swagger
*" | \
  jq -r '.results[].page.url'

# Search for JSON API responses
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com AND page.mime:applica
tion/json" | \
  jq -r '.results[].page.url'

# Search for XML API responses
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com AND page.mime:applica
tion/xml" | \
  jq -r '.results[].page.url'

# Extract API calls from a specific scan's network requests
# (requires scan UUID from a target page scan)
curl -s "https://urlscan.io/api/v1/result/$uuid/" | \
  jq -r '.data.requests[]?.request?.url' | grep -i "api" | sort -u
```

5.3 Hunting Third-Party Integrations

```
# Find all third-party services a target uses
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com&size=100" | \
  jq -r '.results[].lists.linkDomains[]?.domain' | sort -u

# Find S3 buckets referenced by target
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com&size=100" | \
  jq -r '.results[].lists.linkDomains[]?.domain' | grep "amazonaws.com" | sort -u

# Find Google Cloud resources
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com&size=100" | \
  jq -r '.results[].lists.linkDomains[]?.domain' | grep "googleapis.com\|googleuserc
ontent.com" | sort -u

# Find Cloudflare resources
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com&size=100" | \
  jq -r '.results[].lists.linkDomains[]?.domain' | grep "cloudflare" | sort -u

# Find Firebase resources
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com&size=100" | \
  jq -r '.results[].lists.linkDomains[]?.domain' | grep "firebase" | sort -u

# Extract tracking/analytics services
curl -s "https://urlscan.io/api/v1/result/$uuid/" | \
  jq -r '.data.requests[]?.request?.url' | grep -E "google-analytics|segment|mixpane
l|amplitude|hotjar" | sort -u
```

> **Third-Party Insight**
>
> Third-party integrations revealed in URLScan network logs can indicate potential supply chain vulnerabilities. A target using an outdated CDN version, an exposed S3 bucket, or a misconfigured Firebase instance could be reported if it leaks target data.

6. Historical Data Analysis

```
# Find oldest scans for a domain (historical recon)
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com&sort=date" | \
  jq -r '.results[] | "\(.task.time) - \(.task.url)"' | head -20

# Find scans from specific time period
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com AND date:[2024-01-01
TO 2024-06-01]" | \
  jq -r '.results[].page.url'

# Compare technology changes over time
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com&sort=date&size=50" |
\
  jq -r '.results[] | "\(.task.time) - \(.meta.processors?.wappalyzer?.data?.applica
tions[0]?.name // "N/A")"'

# Find discontinued/deleted pages that were previously scanned
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com AND page.status:404"
| \
  jq -r '.results[].task.url'

# Find redirect chains that changed over time
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com AND page.status:301"
| \
  jq -r '.results[] | "\(.task.url) -> \(.page.url)"'

# Extract all page titles for content analysis
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com&size=1000" | \
  jq -r '.results[].page.title' | sort -u
```

Historical Page Title Analysis

Page titles often reveal functionality changes, admin additions, or feature removals over time.

```
# Extract and analyze all unique page titles
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com&size=1000" | \
  jq -r '.results[] | "\(.task.time) | \(.page.title) | \(.page.url)"' | \
  grep -iE "admin|dashboard|internal|api|debug|beta|test|dev|staging" | sort
```

Passive urlscan

7. Integration with Recon Pipelines

7.1 URLScan as a URL Source

```
# Full pipeline: URLScan -> subdomains -> live check -> crawling
# Step 1: Extract subdomains from URLScan
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com&size=10000" | \
  jq -r '.results[].page.domain' | sort -u | \
  tee urlscan_subs.txt | \
  # Step 2: Verify live hosts
  httpx -silent -mc 200,301,302 | \
  tee live_hosts.txt | \
  # Step 3: Crawl for endpoints
  katana -list - -jc -d 3 | \
  tee endpoints.txt | \
  # Step 4: Archive mining
  gau --subs | \
  sort -u > all_target_urls.txt

# URLScan -> waybackurls -> katana -> ffuf pipeline
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com&size=1000" | \
  jq -r '.results[].task.url' | sort -u | \
  waybackurls | \
  katana -list - -jc | \
  httpx -silent -mc 200 | \
  ffuf -w - -u FUZZ -mc 200 -t 50
```

7.2 URLScan for Technology-Driven Targeting

```
# Find all sites using a vulnerable technology version
curl -s "https://urlscan.io/api/v1/search/?q=page.server:Apache/2.4.49&size=1000" |
\
  jq -r '.results[].page.domain' | sort -u

# Find all sites running outdated WordPress
curl -s "https://urlscan.io/api/v1/search/?q=page.title:WordPress&size=100" | \
  jq -r '.results[] | "\(.page.domain) - \(.meta.processors?.wappalyzer?.data?.appli
cations[]? | select(.name=="WordPress") | .version)"'

# Find all Django sites (check for CSRF cookie pattern in scans)
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com" | \
  jq -r '.results[] | select(.data.cookies[]?.name == "csrftoken") | .page.url'
```

7.3 Automated Monitoring

```
# Monitor target for new URLs discovered by URLScan
#!/bin/bash
TARGET="target.com"
SEEN_FILE="seen_uuids.txt"
touch "$SEEN_FILE"

while true; do
  # Get recent scans
  results=$(curl -s "https://urlscan.io/api/v1/search/?q=domain:$TARGET&size=100")

  echo "$results" | jq -r '.results[]' | while read -r result; do
    uuid=$(echo "$result" | jq -r '._id')
    url=$(echo "$result" | jq -r '.page.url')

    if ! grep -q "$uuid" "$SEEN_FILE"; then
      echo "NEW SCAN: $uuid -> $url"
      echo "$uuid" >> "$SEEN_FILE"

      # Extract and process the new scan
      curl -s "https://urlscan.io/api/v1/result/$uuid/" | \
        jq -r '.data.requests[]?.request?.url' >> discovered_urls.txt
    fi
  done

  sleep 3600  # Check every hour
done
```

8. One-Liners & Automation

Extract All Subdomains from URLScan

```
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com&size=10000" | \
  jq -r '.results[] | [.task.url, .page.url, .page.domain] | .[]' | \
  grep -oE '[a-zA-Z0-9][a-zA-Z0-9.-]*\.target\.com' | sort -u
```

Passive urlscan

Extract API Endpoints from All Target Scans

```
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com&size=100" | \
  jq -r '.results[]._id' | while read uuid; do
    curl -s "https://urlscan.io/api/v1/result/$uuid/" | \
    jq -r '.data.requests[]?.request?.url' | grep -i "api"
  done | sort -u
```

Passive urlscan

Find Exposed Admin Panels on Target

```
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com AND page.title:ad
min" | \
  jq -r '.results[] | "\(.page.url) - \(.page.title)"' | sort -u
```

Passive High Value

Technology + Version Enumeration

```
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com&size=100" | \
  jq -r '.results[].meta.processors?.wappalyzer?.data?.applications[]? | "\(.nam
e):\(.version // "unknown")"' | \
  sort | uniq -c | sort -rn
```

Passive urlscan

Full URLScan Recon Pipeline

```
# Complete target recon using URLScan as primary source
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com&size=10000" | \
  jq -r '.results[] | [.page.domain, .page.ip, .page.asn, .page.asnname, .page.s
erver] | @tsv' | \
  sort -u | tee urlscan_recon.tsv

# Extract just the network infrastructure
curl -s "https://urlscan.io/api/v1/search/?q=domain:target.com&size=10000" | \
  jq -r '.results[] | "\(.page.ip) - \(.page.asn) - \(.page.asnname)"' | sort -u
```

Passive urlscan

Submit and Analyze Target URLs in Bulk

```
# Submit multiple URLs for scanning and collect results
cat target_urls.txt | while read url; do
  result=$(curl -s -X POST "https://urlscan.io/api/v1/scan/" \
    -H "Content-Type: application/json" \
    -H "API-Key: $URLSCAN_API_KEY" \
    -d "{\"url\":\"$url\",\"public\":\"on\",\"tags\":[\"bugbounty\"]}")
  uuid=$(echo "$result" | jq -r '.uuid')
  echo "Submitted: $url -> UUID: $uuid"
  echo "$uuid" >> submitted_uuids.txt
done

# After waiting for scans to complete, process results
cat submitted_uuids.txt | while read uuid; do
  curl -s "https://urlscan.io/api/v1/result/$uuid/" | \
    jq -r '"URL: \(.task.url)\nTitle: \(.page.title)\nTech: \(.meta.processors?.
wappalyzer?.data?.applications[]?.name // "N/A")\n---"'
done
```

Active urlscan

9. Tips, Tricks & Pitfalls

Tip 1: Combine with Certificate Transparency

URLScan only shows scanned URLs. Combine with crt.sh and Censys for complete subdomain coverage. Scans submitted by other researchers may reveal subdomains not found in certificate logs.

Tip 2: Monitor for New Scans

Run automated queries daily to catch new scans submitted by other users or your own monitoring. New scans on your target may reveal recently discovered endpoints.

Tip 3: Focus on Page Titles

Page titles are highly searchable and reveal a lot: "404 Not Found" vs "Dashboard" vs "API Documentation". Use title-based queries to quickly find interesting pages.

Tip 4: ASN Infrastructure Mapping

Once you identify the target's ASN, search URLScan for all other domains on that ASN. You may discover acquisitions, subsidiaries, or related services in scope.

Tip 5: Extract Redirect Chains

URLScan records the full redirect chain. Analyzing chains can reveal open redirects, SSO flows, and URL parameters that persist across redirects.

Tip 6: Cookie and Header Analysis

Scan results include all cookies set and response headers. Look for: internal cookie names revealing technology, missing security headers, Set-Cookie on unexpected domains.

Pitfall 1: Public Scans Expose Your Recon

Remember that scans you submit are visible to everyone, including the target. If you're hunting on a program that monitors URLScan, your reconnaissance targets are exposed.

Pitfall 2: Rate Limits

Without an API key, URLScan limits searches and submissions. Sign up for a free API key to increase limits. Paid plans offer higher throughput for bulk reconnaissance.

Pitfall 3: Scans May Be Stale

A scan from 6 months ago may show endpoints that no longer exist. Always verify discovered URLs with a live check (httpx/curl) before reporting or investing deep testing time.

Pitfall 4: Not All URLs Are Scanned

URLScan only knows what has been submitted by users. Internal endpoints, authenticated pages, and obscure URLs are unlikely to appear. Use it as one of many recon sources.

10. Tools Comparison

Tool	Type	Strengths	Limitations
URLScan.io	Web Sandbox + Search	Screenshots, network requests, technology detection, historical data	Public scans only, rate limited, user-submitted coverage gaps
Wayback Machine	Web Archive	Very deep historical data, page snapshots	No network request data, no technology detection
Common Crawl	Web Crawl Corpus	Massive dataset, raw crawl data	Difficult to query, no screenshots, delayed updates
Censys	Host/Port Scan	Certificate data, port scanning, service detection	No page content or screenshots
Shodan	Host/Port Scan	IoT/embedded devices, banner grabbing, exposed services	Web application content limited
URLScan Search	URL Database	Network request chains, client-side behavior, cookies	Only covers submitted URLs

> **Best Practice**
>
> Use URLScan.io as part of a layered reconnaissance strategy: passive sources (URLScan, Wayback, CT logs) for seed URL discovery, followed by active crawling (Katana, Gospider) for endpoint expansion, then targeted scanning (ffuf, feroxbuster) for hidden paths. URLScan's unique value is in its network request logging and technology detection - capabilities not found in purely DNS or archive-based tools.